



3

Backing Up Your WhatsPackt Messages

In any chat application, data handling is a significant concern – we need to ensure that messages sent and received are stored correctly, quickly retrieved when needed, and resilient to potential losses due to unforeseen circumstances such as device failures or accidental deletions. This requires a robust data persistence strategy. We also need to consider performance and user experience, which calls for effective caching mechanisms, as well as making sure that we have backups in the event of data loss or when the user changes devices.

In this chapter, we will start by introducing you to Room, a persistence library that provides an abstraction layer over SQLite and makes it easier to work with databases in Android. You'll learn about its architecture and components and how to use it to store and retrieve chat conversations and messages.

Next, we will tackle the creation of a cache mechanism orchestrating the use of Room locally and the use of the API to gather data from the backend.

Moving forward, we'll get you up to speed with Firebase Storage. You'll learn to set it up, understand its benefits, and how to secure data stored in it. We'll then use Firebase Storage to create a backup of our chat conversations, an essential feature for any chat application.

Finally, we'll explore how to use **WorkManager**, an API that makes it easy to schedule deferrable, asynchronous tasks even if the app exits or the device restarts. You'll learn how it can be used to schedule chat backups and how to upload these backups to **Amazon Simple Storage Service (Amazon S3)**, ensuring data safety.

So, in this chapter, we will be covering the following topics:

- Understanding Room
- Implementing Room in WhatsApp
- Getting to know Firebase Storage
- Scheduling **WorkManager** to send backups
- Using Amazon S3 for storage

Technical requirements

As in the previous chapter, you will need to have installed Android Studio (or another editor of your preference).

We are also going to assume that you followed along with the previous chapter. You can download this chapter's complete code from here:

<https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-3>.

Understanding Room

When it comes to Android development, one of the most essential tasks is managing your application's data in a local database. The **Room** persistence library, part of Android Jetpack, is an abstraction layer over SQLite, a popular database that comes with Android. Room offers more robust database access while harnessing SQLite's full power.

Key features of Room

Before Room, developers primarily used **SQLite** directly or other **object-relational mapping (ORM)** libraries. While SQLite is powerful, it can be cumbersome to work with because it requires writing a lot of boilerplate code. Additionally, errors in SQL queries often aren't detected until runtime, which can lead to crashes.

Room solves these issues by providing a simpler and more robust API over the standard SQLite for managing local data storage. Here are some of its key features:

- **Compile-time verification of SQL queries:** Room verifies your SQL queries at compile time, not at runtime. This means if there's an error in one of your queries, you'll know as soon as you compile your app, not after you've shipped it to users. This leads to more robust and reliable code.
- **Reduced boilerplate code:** With Room, you don't need to write as much code to perform simple database operations. This leads to cleaner, more readable code.
- **Integration with other architecture components:** Room is designed to integrate seamlessly with other **Android Architecture Components (AAC)** library components, such as **LiveData** and **ViewModel**. This means you can create a well-architected, robust app that follows best practices for Android development.
- **Easy migration paths:** Room offers robust migration support, including migration paths and testing. As your app's data needs evolve, Room makes it easy to adapt your database structure to meet those needs.
- **Supports complex queries:** Despite simplifying interaction with SQLite, Room still allows you to perform complex SQL queries when you need more flexibility and power.

As you can see, Room offers an efficient and streamlined approach to managing your app's local data. It's a powerful tool that can make your Android development experience much more pleasant and productive.

Room's architecture and components

Room's architecture is based on three main components:

- **Database**
- **Entity**
- **Data Access Object (DAO)**

Here, you can see how every Room component interacts with the rest of the app:

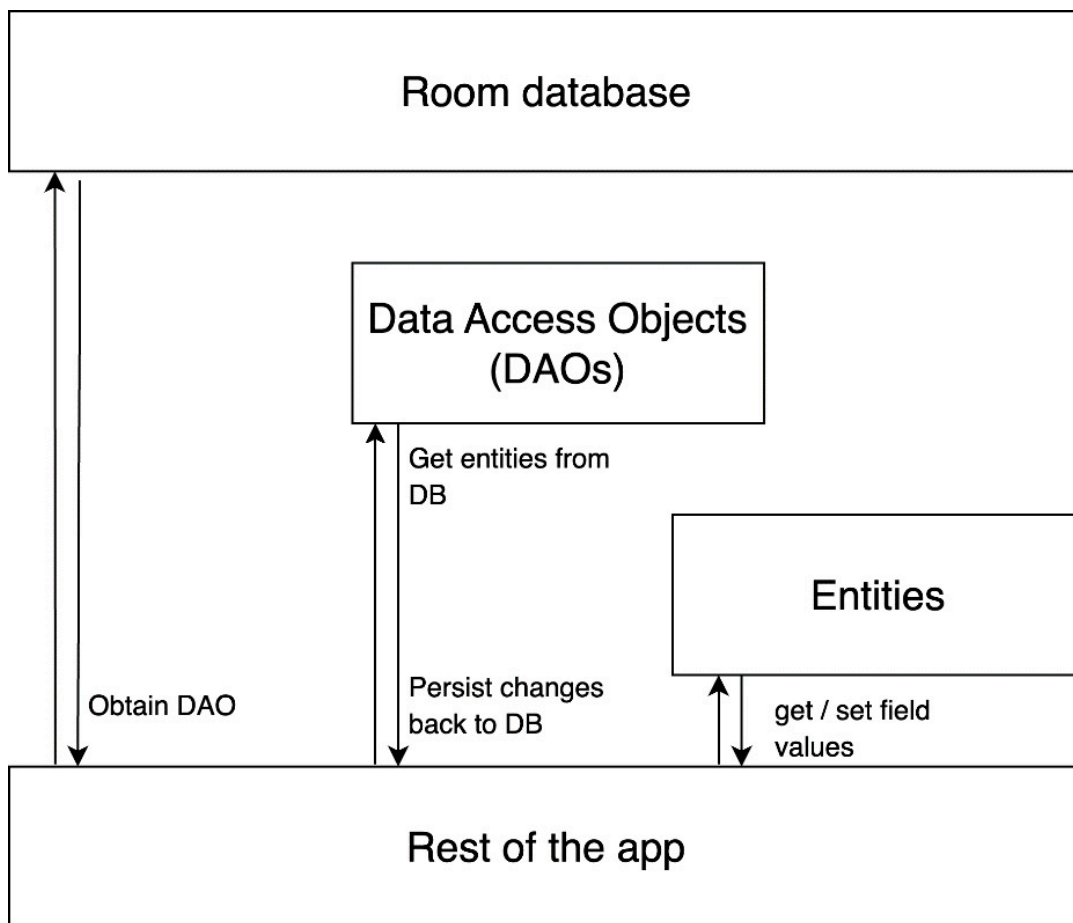


Figure 3.1: Diagram of Room architecture

Understanding these components is crucial when using Room effectively, so let's dive into them deeper.

Database

The **Database** class in Room is a high-level class that works as the main access point to your app's persisted data. It's an abstract class where you define an abstract method for each **@Dao** annotation in your app. When you create an instance of the **Database** class, Room generates the implementation code of these DAO methods (DAO will be explored in more detail in a moment).

The **Database** class is annotated with **@Database**, specifying the entities it comprises and the database version. If you modify the database schema, you need to update the version number and define a migration strategy, as in the following example:

```
@Database(entities = [Message::class, Conversation::class],
    version = 1)
abstract class ChatAppDatabase : RoomDatabase() {
    abstract fun messageDao(): MessageDao
    abstract fun conversationDao(): ConversationDao
}
```

Here, we've defined a **ChatAppDatabase** Room **Database** class with two entities, **Message** and **Conversation**. We've also defined abstract methods to access our DAOs – **messageDao()** and **conversationDao()**. The **entities** parameter in the **@Database** annotation takes an array of all entities in the database, while the **version** parameter is used for database migration purposes.

Entity

Entities in Room represent the tables in a database. Each entity corresponds to a table, and each instance of an entity represents a row in the table. Room uses the class fields in an entity to define the columns in a table.

You declare an entity by annotating a data class with **@Entity**. Each **@Entity** class represents a table in your database, and you can define

the table name. If you don't define a table name, Room uses the class name as the table name, as in the following example:

```
@Entity(tableName = "messages")
data class Message(
    @PrimaryKey val id: String,
    @ColumnInfo(name = "conversation_id") val
        conversationId: String,
    // ...
)
```

Here, **Message** is an entity that represents a **"messages"** table in our database. Each instance of **Message** will represent a row within the **"messages"** table. Each property in the **Message** class represents a column in the table. The **@PrimaryKey** annotation is used to denote a primary key, and the **@ColumnInfo** annotation is used to specify the column name in the database. If not specified, Room uses the variable name as the column name.

DAO

DAOs are interfaces that define all the database operations that you want to perform. For each DAO, you can define methods for different operations such as insertion, deletion, and querying.

You should annotate an interface with **@Dao**, and then annotate each method with the corresponding operation you want to perform, such as **@Insert**, **@Delete**, **@Update**, or **@Query** for custom queries. Then, Room will autogenerate the necessary code to perform these operations at compile time. Here's an example:

```
@Dao
interface MessageDao {
    @Insert
    fun insert(message: Message)
```

```
@Query("SELECT * FROM messages WHERE conversation_id =  
        :conversationId")  
fun getMessagesForConversation(conversationId: String):  
    List<Message>  
}
```

In this **MessageDao** interface, we've defined two methods – **insert()** for inserting a **Message** object into our database and **getMessagesForConversation()** to retrieve all messages related to a specific conversation from our database. The **@Insert** annotation is a convenience annotation for inserting an entity into a table. The **@Query** annotation allows us to write SQL queries to perform complex reads and writes.

Understanding these components will allow us to leverage the power of Room effectively. The following sections will guide you through the process of implementing Room in our WhatsApp application, starting from setting it up in Android Studio to creating entities and DAOs.

Implementing Room in WhatsApp

In this section, you will be guided through the practical steps of implementing Room in our chat application. We will begin by setting up Room in Android Studio, followed by creating entities and DAOs and eventually using these components to interact with our database.

Adding dependencies

To start using Room, we first need to include the necessary dependencies in our project. Open your **build.gradle** file and add the following dependencies under **dependencies**:

```
dependencies {  
    implementation "androidx.room:room-runtime:2.3.0"
```

```
kapt "androidx.room:room-compiler:2.3.0"
implementation "androidx.room:room-ktx:2.3.0"
// optional - Test helpers
testImplementation "androidx.room:room-testing:2.3.0"
}
```

The **room-runtime** dependency includes the core Room library, while the **room-compiler** dependency is required for Room's annotation-processing capabilities. Room's Kotlin extensions and coroutines support are provided by **room-ktx**, while **room-testing** provides useful classes for testing your Room setup.

After adding these lines, sync your project. You can do it using the **File | Sync Project with Gradle Files** option from the Android Studio menu or by selecting **Sync Now** in the automatic message that appears in the editor after adding the dependencies to the **build.gradle** file:



Figure 3.2: The Sync Now option that appears in Android Studio when it detects any changes to Gradle files

We are ready now to create our database.

Creating the database

As discussed before, the **Database** component is the main access point for our app's data. So, let's create a **ChatAppDatabase** class:

```
@Database(entities = [Message::class, Conversation::class],
version = 1)
abstract class ChatAppDatabase : RoomDatabase() {
    abstract fun messageDao(): MessageDao
    abstract fun conversationDao(): ConversationDao
    companion object {
        @Volatile
        private var INSTANCE: ChatAppDatabase? = null
    }
}
```

```

    fun getDatabase(context: Context): ChatAppDatabase {
        return INSTANCE ?: synchronized(this) {
            val instance = Room.databaseBuilder(
                context.applicationContext,
                ChatAppDatabase::class.java,
                "chat_database"
            ).build()
            INSTANCE = instance
            instance
        }
    }
}

```

The **@Database** annotation marks this class as a Room database. It takes two parameters:

- **entities** is an array of classes that are annotated with **@Entity**, representing the tables within the database. In this case, the **Message** and **Conversation** classes are entities of **ChatAppDatabase**.
- **version** is the database version. If you make changes to the database schema, you'll need to increment this version number and define a migration strategy.

Next, **abstract fun messageDao(): MessageDao** and **abstract fun conversationDao(): ConversationDao** are abstract methods that return the respective DAOs. They do not have method bodies because Room generates their implementations.

Then, we declare a companion object to hold a singleton instance of **ChatAppDatabase**, by using the **@Volatile** annotation. This annotation means **INSTANCE** can be accessed by multiple threads at once but always in a consistent state, meaning a change made by one thread to **INSTANCE** is immediately visible to all other threads. **INSTANCE** is marked as nullable because it might not be initialized immediately.

In the `getDatabase()` function, we're implementing a common pattern for creating a singleton instance of a class in a thread-safe way. This pattern ensures that only one instance of `ChatAppDatabase` is ever created.

We use the `?:` operator to check whether `INSTANCE` is not `null`, and if it is, we enter the synchronized block. This block ensures that only one thread can enter this block of code at a time, preventing the creation of multiple instances of `ChatAppDatabase` if the function is called concurrently from multiple threads.

Within the synchronized block, we're calling `Room.databaseBuilder()` to create a new instance of `ChatAppDatabase`. We provide the application context to avoid memory leaks, the class of the database, and the name of the database.

Finally, we call `build()` to create the `ChatAppDatabase` instance.

After creating the new instance, we assign it to `INSTANCE` to cache it and then return the instance. The next time `getDatabase` is called, it will return the cached database instance instead of creating a new one. This is important because creating a Room database instance is an expensive operation, and having multiple instances would be a waste of resources.

This structure is essential for creating a database instance that will allow us to store messages and conversations.

The next step is to create entity classes.

Creating entity classes

The first entity class we are going to create is the `Message` class:

```
@Entity(
    tableName = "messages",
    foreignKeys = [
        ForeignKey(
            entity = Conversation::class,
            parentColumns = arrayOf("id"),
            childColumns = arrayOf("conversation_id"),
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [
        Index(value = ["conversation_id"])
    ]
)
data class Message(
    @PrimaryKey(name = "id") val id: Int,
    @ColumnInfo(name = "conversation_id") val
        conversationId: Int,
    @ColumnInfo(name = "sender") val sender: String,
    @ColumnInfo(name = "content") val content: String,
    @ColumnInfo(name = "timestamp") val timestamp: Long
)
```

In this code, we are including quite a lot of instructions in the annotations, so let's go through them.

The **@Entity** annotation tells Room to treat this class as a table in the database. It comes with optional arguments, some of which are used here:

- **tableName**: This sets the name of the table as it will appear in the database. In this case, our table will be named **"messages"**.
- **foreignKeys**: This sets up a foreign key relationship with another table. A **ForeignKey** instance takes four main arguments:
 - **entity**: This represents the class of the parent table that this entity has a relationship with. In this case, it's **Conversation::class**.

- **parentColumns:** This specifies the column(s) in the parent entity that the foreign key references. Here, it's the **id** field of **Conversation**.
- **childColumns:** This specifies the column(s) in the child entity that holds the foreign key. Here, it's the **conversation_id** field in **Message**.
- **onDelete:** This represents the action that will be taken if the referenced row in the parent table is deleted. Here, **ForeignKey.CASCADE** is used, which means that if a **Conversation** instance is deleted, all messages that have a **conversation_id** value referencing the conversation's ID will be deleted as well.
- **indices:** This is used to create an index on **conversation_id** to speed up your queries. An index makes data retrieval faster at the cost of additional disk space and slower write speed. An index is particularly useful here because we will often perform operations related to a specific conversation, and indexing **conversation_id** will make these operations more efficient.

Then, we have also added annotations to the properties of the class:

- **@PrimaryKey:** This annotation indicates that the **id** field is the primary key for the **Message** table. A primary key uniquely identifies each row in the table. We could use here **autoGenerate = true**, which means that this field will be automatically filled with an incrementing integer for each new row.
- **@ColumnInfo(name = "column_name"):** This annotation lets you specify a custom column name in the database. If not specified, Room will use the variable name as the column name.

Now, let's create a **Conversation** entity:

```
@Entity(  
    tableName = "conversations",
```

```
)  
class Conversation(  
    @PrimaryKey  
    @ColumnInfo(name = "id") val id: String,  
    @ColumnInfo(name = "last_message_time") val  
        lastMessageTime: Long  
)
```

The **Conversation** entity is very simple – we will just store the **Conversation** ID and the time of the last message in the conversation.

Now that we have created and defined our entities, it's time to create DAOs in order to obtain and update data.

Creating DAOs

A DAO is an interface that serves as a communication layer between the application code and the database. It defines methods for each operation we might perform on the entities in our database.

Let's start with the DAO for the **Message** entity:

```
@Dao  
interface MessageDao {  
    @Query("SELECT * FROM messages WHERE conversation_id =  
        :conversationId ORDER BY timestamp ASC")  
    fun getMessagesInConversation(conversationId: Int):  
        Flow<List<Message>>  
    @Insert(onConflict = OnConflictStrategy.REPLACE)  
    suspend fun insertMessage(message: Message): Long  
    @Delete  
    suspend fun deleteMessage(message: Message)  
}
```

Breaking down the code, we have the following:

- **@Dao**: This annotation identifies the interface as a DAO.

- **@Query**: This annotation is used to specify SQL statements for complex data retrieval tasks.
- **@Insert**: This annotation is used to define a method that inserts its argument into the database. **OnConflictStrategy.REPLACE** means that if a message with the same primary key already exists, it will be replaced by a new one.
- **@Delete**: This annotation is used to define a method that deletes its argument from the database.

Now, let's create a DAO for the **Conversation** entity:

```
@Dao
interface ConversationDao {
    @Query("SELECT * FROM conversations ORDER BY
        last_message_time DESC")
    fun getAllConversations(): Flow<List<Conversation>>
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertConversation(conversation:
        Conversation): Long
    @Delete
    suspend fun deleteConversation(conversation:
        Conversation)
}
```

The annotations function the same way as they did in **MessageDao**. Here, we're retrieving all conversations ordered by the time of their last message, and we have methods for inserting and deleting conversations.

We now need to provide these DAOs for other app components so that they can be injected. With that in mind, we will create the following module:

```
@Module
@InstallIn(SingletonComponent::class)
```

```
object DatabaseModule {
    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext appContext:
        Context): ChatAppDatabase {
        return ChatAppDatabase.getDatabase(appContext)
    }
    @Provides
    fun provideMessageDao(database: ChatAppDatabase):
        MessageDao {
        return database.messageDao()
    }
    @Provides
    fun provideConversationDao(database: ChatAppDatabase):
        ConversationDao {
        return database.conversationDao()
    }
}
```

As we have previously covered the creation of Hilt modules in the previous chapters, we won't go over all the code again. Instead, here are the key parts of the code:

- We are using **@Singleton** to indicate that only a single instance of the object should be created and provided as a dependency.
- The **@ApplicationContext** qualifier tells Hilt that we want to inject the **Application Context** into the method. This is quite useful as, in Android, we have **Application Context** and **Activity Context** which have different lifecycles. Remember that a **Context** in Android is an interface to global information about an application environment, offering access to resources and system services and existing in various scopes such as **Application Context**, which is tied to the lifecycle of the application, and **Activity Context**, which is associated with the lifecycle of an activity, among others. If we don't specify which **Context** we want to use, we can get confused or provide one that is not suitable for this situation. Using the **@ApplicationContext** qualifier will assure us that the **Context** in-

jected will be the expected one (the **Application Context**, in this case).

Now, as we already did in the previous chapter for the API or WebSocket, we are going to create a data source to connect with the database: **LocalMessagesDataSource**.

Creating a LocalMessagesDataSource data source

We need to create a **LocalMessagesDataSource** data source that will wrap the DAO and expose the specific database operations our app needs. This way, if we decide to change the database in the future, we will only have to change it here (not in the rest of consumers). This class will serve as a DAO at a higher level of abstraction, simplifying the API for the rest of our app and making it easier to mock the database in tests.

In the following code, we are just calling the functions we already defined in the DAO:

```
class MessagesLocalDataSource @Inject constructor(private
val messageDao: MessageDao) {
    fun getMessagesInConversation(conversationId: Int):
    Flow<List<Message>> {
        return
            messageDao.getMessagesInConversation(
                conversationId)
    }
    suspend fun insertMessage(message: Message): Long {
        return messageDao.insertMessage(message)
    }
    suspend fun deleteMessage(message: Message) {
        messageDao.deleteMessage(message)
    }
}
```

```
}  
}
```

As we said before, we will use this data source to wrap the database and provide an additional abstraction layer.

Now, it's time to combine this local data source with the remote one. This will force us to think about a caching strategy.

Handling two data sources in the **MessagesRepository** component

Up until now, we only had one data source (the WebSocket one), but we would like our users to be able to retrieve their messages even if they have no connection for a short time. That's the reason why we have just created a database and have it ready to be populated.

As our use case is to provide a fallback for the WebSocket so that the user can continue checking their messages, we will follow a strategy where the main source of truth will continue being the WebSocket, but we will store a copy of the messages in the app database. Also, we don't want the records of this database to grow infinitely, so we are setting a cap of **100** messages per conversation.

The component responsible for combining both data sources is **MessagesRepository**, which we already implemented to be connected to **WebsocketDataSource** in the previous chapter. Let's now modify it to include both data sources and to orchestrate the data retrieval and local storage:

```
class MessagesRepository @Inject constructor(  
    private val dataSource: MessagesSocketDataSource,  
    private val localDataSource: DatabaseDataSource  
): IMessagesRepository {
```

Next, we will modify the **getMessages()** method to include the logic to store the information retrieved from **MessagesSocketDataSource** (remote data source) in **DatabaseDataSource** (local data source):

```
override suspend fun getMessages(chatId: String, userId:
String): Flow<Message> {
    return flow {
        try {
            dataSource.connect().collect { message ->
                localDataSource.insertMessage(message)
                emit(message)
                manageDatabaseSize()
            }
        } catch (e: Exception) {
            localDataSource.getMessagesInConversation(
                chatId.toInt()).collect {
                it.forEach { message -> emit(message) }
            }
        }
    }
}
```

As can be seen, we have connected to the socket data source, but we have wrapped this action in a **try-catch** block. So, if everything goes correctly, we will store in our database every new message and then emit it in the flow.

At the same time, we call **manageDatabaseSize()**, which will check and keep the size of the database under the limit we have set (100 maximum messages per conversation). If the socket fails, we will retrieve messages from the database directly.

Now, we will also modify the **sendMessage** method, where we will also store every new message sent:

```
override suspend fun sendMessage(chatId: String,
message: Message) {
    dataSource.sendMessage(message)
    localDataSource.insertMessage(message)
}
```

The disconnect will be kept the same as we don't need to do anything related to the new data source:

```
override suspend fun disconnect() {
    dataSource.disconnect()
}
```

Finally, here is the mechanism that we will implement to keep the size of the database under the agreed number of messages per conversation:

```
private suspend fun manageDatabaseSize() {
    val messages =
        localDataSource.getMessagesInConversation(
            chatId.toInt()).first()
    if (messages.size > 100) {
        // Delete the oldest messages until we have 100
        left
        messages.sortedBy { it.timestamp
        }.take(messages.size - 100).forEach {
            localDataSource.deleteMessage(it)
        }
    }
}
```

We will obtain all messages related to the conversation and check if the size is more than 100. Then, we will order them based on their timestamp and remove the oldest ones.

Now, we have the Room database integrated into our app. Our last messages will be available even if we lose the connection. In the following section, let's see how we can also send a backup of those messages to be stored in the cloud. For that, we will use Firebase Storage.

Getting to know Firebase Storage

Firebase Storage, also known as Cloud Storage for Firebase, is a powerful object storage service built for Google scale. It enables developers to store and retrieve user-generated content, such as photos, videos, or other forms of user data. Firebase Storage is backed by **Google Cloud Storage (GCS)**, making it robust and scalable for any size of data, from small text files to large video files.

Here are some of the key features and capabilities of Firebase Storage:

- **User-generated content:** Firebase Storage allows your users to upload their own content directly from their devices. This could include anything from profile pictures to blog posts.
- **Integration with Firebase and Google Cloud:** Firebase Storage integrates smoothly with the rest of the Firebase ecosystem, including Firebase Authentication and Firebase Security Rules. It's also a part of the larger Google Cloud ecosystem, which opens up possibilities for using Google Cloud's advanced features, such as Cloud Functions.
- **Security:** Firebase Storage provides robust security features. Using Firebase Security Rules, you can control who has access to what data. You can restrict access based on a user's authentication state, identity, and claims, as well as data patterns and metadata.
- **Scalability:** Firebase Storage is designed to handle a large number of uploads, downloads, and storage of data. It automatically scales with your user base and traffic, meaning you don't need to worry about capacity planning.

- **Offline capabilities:** Firebase **software development kits (SDKs)** for Cloud Storage add Google security to file uploads and downloads for your Firebase apps, regardless of network quality. You can use it to pause, resume, and cancel transfers.
- **Rich media:** Firebase Storage supports rich media content. This means you can use it to store images, audio, video, or even other binary data.
- **Strong consistency:** Firebase Storage guarantees strong consistency, meaning that once an upload or download is completed, the data is immediately available from all Google Cloud Storage locations, and any subsequent reads will return the latest updated data.

In our context, a messaging application, Firebase Storage could be used to store and retrieve message history or backups, shared files, or even multimedia content within conversations. This could serve as a reliable backup solution or a means of synchronizing chat history across multiple devices. However, you need to ensure you handle privacy and security concerns, especially since chat conversations can contain sensitive data.

How Firebase Storage works

In Firebase Storage, data is stored as objects within a hierarchical structure. The full path to an object in Firebase Storage includes the project ID and the object's location within the storage bucket.

NOTE

*In the context of cloud storage, a **bucket** is a basic container that holds data. It's the primary parent in the hierarchy of data organization. All data in cloud storage is stored in buckets. The concept of a bucket is used by many cloud storage systems, including GCS, Amazon S3, and Firebase Storage. These systems typically allow you to create one or more buckets in your storage space and then upload data as objects or files to these buckets. Each bucket has a unique name within the cloud*

storage system, and it contains data objects, or files, each of which is identified by a key or a name.

The object's location is defined by a path that you specify. This path is similar to a filesystem path and includes both the directories and the filename. For example, in the **images/profiles/user123.jpg** path, **images** and **profiles** are directories, and **user123.jpg** is the filename.

When you upload a file to Firebase Storage, you create a reference to the location where you're going to store the file. This reference is represented by a **StorageReference** object, which you create by calling the **child()** method on a reference to your Firebase Storage bucket and passing the path as an argument, as in the following example:

```
val storageRef = Firebase.storage.reference
val fileRef =
    storageRef.child("images/profiles/user123.jpg")
```

Here, **fileRef** is a reference to the **user123.jpg** file in the profile's directory within the **images** directory.

You can use this reference to perform various operations, such as uploading a file, downloading a file, or getting a URL to the file. Each of these operations returns a **Task** object that you can use to monitor the operation's progress or get its result.

The paths in Firebase Storage are flexible, and you can structure them in a way that makes sense for your application. For instance, in a messaging application, you might store conversation logs in a **chat_logs** directory, with each log's filename being the chat's ID. The path to a chat log might look like this: **chat_logs/chat123.txt**.

Finally, it's worth noting that Firebase Storage uses rules to control who can read and write to your storage bucket. By default, only au-

thenticated users can read and write data. You can customize these rules to suit your application's needs.

Let's start setting up Firebase Storage in our project.

Setting up Firebase Storage

To start using Firebase Storage, we'll first need to add the Cloud Storage for Firebase Android library to our app. This can be done by adding the following line to our module's **build.gradle** file:

```
implementation 'com.google.firebase:firebase-storage-ktx'
```

As for the chat messages, one approach would be to save the chat logs as text files in Firebase Storage. Each conversation could have its own text file, and each message would be a line in that file. So, we are going to create a data source to upload those files:

```
class StorageDataSource @Inject constructor(private val
firebaseStorage: FirebaseStorage) {
    suspend fun uploadFile(localFile: File, remotePath:
String) {
        val storageRef =
            firebaseStorage.reference.child(remotePath)
        storageRef.putFile(localFile.toUri()).await()
    }
    suspend fun downloadFile(remotePath: String, localFile:
File) {
        val storageRef =
            firebaseStorage.reference.child(remotePath)
        storageRef.getFile(localFile).await()
    }
}
```

Here, we've added the Firebase storage instance as a parameter to the constructor, allowing it to be injected when the class is instantiated using Hilt. The **uploadFile** and **downloadFile** methods suspend the coroutine until the upload or download operation completes, using the **await()** extension function.

To be able to use the Firebase storage instance, we would need to provide the **FirestoreStorage** dependency. For that, we will need to create the following module so that Hilt is aware of how it can obtain it:

```
@Module
@InstallIn(SingletonComponent::class)
object StorageModule {
    @Singleton
    @Provides
    fun provideFirestoreStorage(): FirebaseFirestore =
        FirebaseFirestore.getInstance()
}
```

Now, we need to create those files, to then be uploaded using this data source. We are going to do it in a newly created repository:

BackupRepository.

The **BackupRepository** repository will serve as an intermediary between different data sources (such as local databases via DAOs and remote data sources such as Firebase Storage) and the rest of the application. It retrieves data from the sources, processes it if necessary, and provides it to the calling code in a convenient form.

Here is the code for this repository:

```
class BackupRepository @Inject constructor(
    private val messageDao: MessageDao,
    private val conversationDao: ConversationDao,
    private val storageDataSource: StorageDataSource
```

```
) {  
    private val gson = Gson()  
    suspend fun backupAllConversations() {  
        // Get all the conversations  
        val conversations =  
            conversationDao.getAllConversations()  
        // Backup each conversation  
        for (conversation in conversations) {  
            val messages =  
                messageDao.getMessagesForConversation(  
                    conversation.conversationId)  
            // create a JSON representation of the messages  
            val messagesJson = gson.toJson(messages)  
            // create a temporary file and write the JSON  
            // to it  
            val tempFile = createTempFile("messages",  
                ".json")  
            tempFile.writeText(messagesJson)  
            // upload the file to Firebase Storage  
            val remotePath =  
                "conversations/${conversation.conversationId}  
                }/messages.json"  
            storageDataSource.uploadFile(tempFile,  
                remotePath)  
            // delete the local file  
            tempFile.delete()  
        }  
    }  
    private fun createTempFile(prefix: String, suffix:  
    String): File {  
        // specify the directory where the temporary file  
        // will be created  
        val tempDir =  
            File(System.getProperty("java.io.tmpdir"))  
        // create a temporary file with the specified  
        // prefix and suffix  
        return File.createTempFile(prefix, suffix, tempDir)  
    }  
}
```

As can be seen in the code, it uses **ConversationDao** to fetch all conversations in the local database. Each conversation represents a distinct chat thread.

Then, for each conversation, it fetches the associated messages using **MessageDao**, converts the messages to a JSON string using the Gson library, writes this JSON string to a temporary file, and then uploads the file to Firebase Storage through **StorageDataSource**.

Once the upload to Firebase Storage is complete, it deletes the local temporary file to clean up the storage space on the device.

BackupRepository handles all the details of data retrieval, processing, and storage. Other parts of the application don't need to know how the data is stored or retrieved. They only interact with **BackupRepository**, which provides a simple interface for these operations. This makes the code easier to maintain, understand, and test.

Finally, we will create **UploadMessagesUseCase**, which will be the use case or domain interactor responsible for executing the upload action.

Creating UploadMessagesUseCase

The responsibility of **UploadMessagesUseCase** will be to execute the backup using **BackupRepository**. As most of the logic is already in the repository, the code will be simpler and will look like this:

```
class UploadMessagesUseCase @Inject constructor(
    private val backupRepository: BackupRepository
) {
    suspend operator fun invoke() {
        backupRepository.backupAllConversations()
    }
}
```

Now, we are ready to retrieve and upload these backups. As it can be a time- and resource-consuming task, the idea will be to do it periodically, once per week or once per day. This is where **WorkManager** comes in handy.

Scheduling WorkManager to send backups

WorkManager is a component of Android Jetpack designed to manage and schedule deferrable background tasks. It ensures these tasks run even if the app exits or the device restarts, and effectively handles retries and backoff strategies. As it also takes care of compatibility issues, alongside upholding best practices for battery and system health, **WorkManager** is the recommended tool for tasks that require guaranteed and efficient execution.

WorkManager uses an underlying job dispatching service based on the following criteria:

- It uses **JobScheduler** for devices with API 23 and above
- For devices with API 14 to 22, it uses a combination of **BroadcastReceiver** (for system broadcasts) and **AlarmManager**
- If the app includes the optional **WorkManager** dependency on Firebase **JobDispatcher** and Google Play services are available on the device, **WorkManager** uses Firebase **JobDispatcher**

WorkManager chooses the appropriate way to schedule a background task, depending on the device API level and included dependencies. To use **WorkManager**, we need first to understand how we can create **Worker** and **WorkRequest** instances.

Introducing the Worker class

A **Worker** is a class where you define the task or job that needs to be executed. It is the core class that defines the work that needs to be performed and how to perform that work. You extend the **Worker** class (or **CoroutineWorker** if you're using Kotlin coroutines) and override the **doWork()** method to define what the task should do.

The **doWork()** method is where you put the code that needs to be executed in the background. This is where you define the operation that needs to be performed, such as fetching data from the server, uploading a file, processing an image, and so on.

Each **Worker** instance is given a maximum of 10 minutes to finish its execution and return a **Result** instance. The **Result** instance can be one of three types:

- **Result.success()**: Indicates that the work completed successfully. You can optionally return a **Data** object that can be used as the output data of this work.
- **Result.failure()**: Indicates that the work failed. You can optionally return a **Data** object that can describe the failure.
- **Result.retry()**: Indicates that the work failed and should be tried at another time according to its retry policy.

A unique feature of **Worker** is that it's **lifecycle-aware**. If the task in a **Worker** instance is running and the app goes to the background, the **Worker** instance can continue to run, whereas if the device restarts while the **Worker** instance is running, the task can resume when the device is back up. This ensures that the work will be performed under the constraints specified when creating a **WorkRequest** instance, even if your app process is not around.

Here is an example of a basic **Worker** class:

```
class ExampleWorker(appContext: Context, workerParams:
```

```
WorkerParameters)
    : Worker(appContext, workerParams) {
    override fun doWork(): Result {
        // Code to execute in the background
        return Result.success()
    }
}
```

In the example, we extend the **Worker** class and override the **doWork()** method to specify the task to be performed. In this case, we are just returning the result as successful, but the code to do the actual work would be where the **// Code to execute in the background** comment is placed.

To make our **Worker** instances work, we need another component: **WorkRequest**. Let's see how we can configure and use it.

Configuring the WorkRequest component

WorkRequest is the class that defines an individual unit of work. It encapsulates your **Worker** class, along with any constraints that must be satisfied for the work to run and any input data it needs.

There are two concrete implementations of **WorkRequest** that you can use:

- **OneTimeWorkRequest**: As the name suggests, this represents a one-off job. It will only be executed once.
- **PeriodicWorkRequest**: This is used for repeating jobs that run periodically. The minimum repeat interval that can be defined is 15 minutes. This constraint is discussed further in the official documentation:

<https://developer.android.com/reference/androidx/work/PeriodicWorkRequest>.

WorkRequest has several options for setting conditions for the execution of work and for scheduling multiple pieces of work to run in a particular order:

- **Constraints:** A **WorkRequest** instance can have a **Constraints** object set on it, which allows you to specify conditions that must be met for the work to be eligible to run. For example, you might require that the device is idle or charging, or that it has a certain type of network connectivity. We will learn about these conditions in detail in a few paragraphs.
- **Input data:** You can attach input data to a **WorkRequest** instance using the **setInputData()** method, providing your **Worker** instance with all the information it needs to do its work.
- **Backoff criteria:** You can set backoff criteria for the **WorkRequest** instance to control retry timing when the work fails.
- **Tags:** You can also add tags to your **WorkRequest** instance, which will make it easier to track, observe, or cancel specific groups of work.
- **Chaining work:** **WorkManager** allows you to create dependent chains of work. This means that you can ensure certain pieces of work are executed in a certain order. You can create complex chains that run a series of **WorkRequest** objects in a specific order.

WorkManager offers several types of constraints that you can set on a **WorkRequest** object to specify when your task should run. This is done using the **Constraints.Builder** class. Here are the available constraints you can set:

- **Network type (setRequiredNetworkType):** This constraint specifies the type of network that must be available for the work to run. Options include **NetworkType.NOT_REQUIRED**, **NetworkType.CONNECTED**, **NetworkType.UNMETERED**, **NetworkType.NOT_ROAMING**, and **NetworkType.METERED**.

- **Battery not low** (`setRequiresBatteryNotLow`): If this constraint is set to **true**, the work will only run when the battery isn't low.
- **Device idle** (`setRequiresDeviceIdle`): If this constraint is set to **true**, the work will only run when the device is in idle mode. This is usually when the user hasn't interacted with the device for a period of time.
- **Storage not low** (`setRequiresStorageNotLow`): If set to **true**, the work will only run when the storage isn't low.
- **Device charging** (`setRequiresCharging`): If set to **true**, the work will only run when the device is charging.

Here is an example of how we can configure a **WorkRequest** instance:

```
val constraints = Constraints.Builder()
    .setRequiresCharging(true)
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .setRequiresBatteryNotLow(true)
    .build()
val workRequest = OneTimeWorkRequestBuilder<MyWorker>()
    .setConstraints(constraints)
    .addTag("myWorkTag")
    .build()
```

In this example, **MyWorker** will only run when the device is charging, connected to a network, and the battery level is not low. It will also have a tag, which will allow us to identify it easily.

Here is a diagram with the flow followed for the **Worker** and **WorkRequest** instances to be executed:

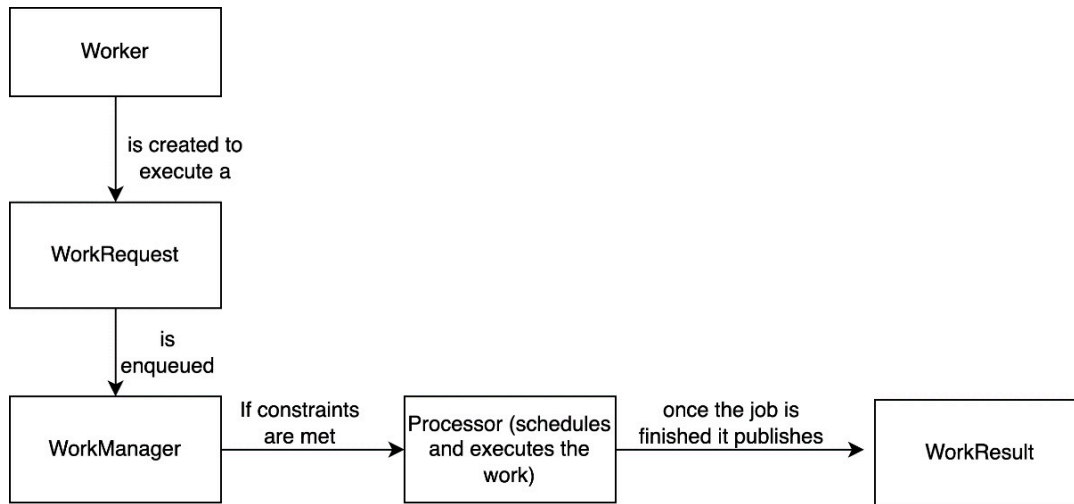


Figure 3.3: Diagram of WorkManager flow to execute a WorkRequest instance

We now have the tools to build our own **Worker** instance and configure the **WorkRequest** instance to retrieve and upload the backup. So, let's actually create them.

Creating our Worker instance

First, to support the **WorkManager** API, we need to include the related dependencies in our code:

```
dependencies {
    implementation "androidx.work:work-runtime-ktx:$2.9.0"
    // Hilt AndroidX WorkManager integration
    implementation 'androidx.hilt:hilt-work:$2.44
    ...
}
```

As we have seen before, the **Worker** class will execute the task that can run in the background even when the app is not being used. In other words, it's a unit of work that can be scheduled to run under certain conditions. In our case, we have just created the logic for that (in **UploadMessagesUseCase**), so our **Worker** class will need to have access to that class.

That's the reason we will start adding **HiltWorker** annotation to our worker. **HiltWorker** is an annotation provided by Hilt's **androidx.hilt** extension library. This annotation tells Hilt that it should create an injectable Worker instance (that is, Hilt should manage the dependencies of this **Worker** instance).

Here's the complete code for our **Worker** class:

```
@HiltWorker
class UploadMessagesWorker @AssistedInject constructor(
    @Assisted appContext: Context,
    @Assisted workerParams: WorkerParameters,
    private val uploadMessagesUseCase:
        UploadMessagesUseCase
) : CoroutineWorker(appContext, workerParams) {
    override suspend fun doWork(): Result = coroutineScope {
        try {
            uploadMessagesUseCase.execute()
            Result.success()
        } catch (e: Exception) {
            if (runAttemptCount < MAX_RETRIES) {
                Result.retry()
            } else {
                Result.failure()
            }
        }
    }
    companion object {
        private const val MAX_RETRIES = 3
    }
}
```

We are also using a new annotation: **AssistedInject**. Now, **AssistedInject** is a Dagger Hilt feature that helps with scenarios where you need to inject some dependencies but also need to provide some arguments at runtime. Here, the **appContext** and **workerParams**

arguments to the constructor are provided at runtime (when the **Worker** instance is created by **WorkManager**), while **uploadMessage-sUseCase** is a dependency that should be injected.

The **doWork()** function is where the work that this **Worker** instance should perform is defined. This function is a suspend function and runs within a coroutine scope. This means it can perform long-running operations such as network requests or database operations without blocking the main thread.

In **doWork()**, **uploadMessagesUseCase.execute()** is called to perform the actual work of uploading messages. If this operation is successful, **Result.success()** is returned. If an **Exception** error is thrown, **Result.retry()** is returned if **runAttemptCount** is less than **MAX_RETRIES**, which means the work should be retried. If **runAttemptCount** equals or exceeds **MAX_RETRIES**, **Result.failure()** is returned, which means the work should not be retried.

As we want it to only retry three times, we are using **runAttemptCount**, which is a property provided by **ListenableWorker** (the superclass of **CoroutineWorker**) that keeps track of how many times the work has been attempted.

Finally, **MAX_RETRIES** is a constant that defines the maximum number of retries. It is set to **3** in this example.

To summarize, this **Worker** instance uploads messages by calling **uploadMessagesUseCase.execute()**, and it can retry the operation up to three times in case of failure. The actual dependencies of this **Worker** instance (**UploadMessagesUseCase**) are provided via **dependency injection (DI)** using Dagger Hilt. Now, we need to set up the **WorkRequest** class.

Setting up the WorkRequest class

In the case of the **WorkRequest** class, we will have to think about how frequently we want our messages to be backed up; for example, we can do a backup once per week. Also, we are going to configure the **WorkRequest** class to be only called when the user has a Wi-Fi connection. Here is how we do it:

```
val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.UNMETERED)
    .build()
val uploadMessagesRequest =
    PeriodicWorkRequestBuilder<UploadMessagesWorker>(7,
        TimeUnit.DAYS)
        .setConstraints(constraints)
        .setBackoffCriteria(BackoffPolicy.LINEAR,
            PeriodicWorkRequest.MIN_PERIODIC_INTERVAL_MILLIS,
            TimeUnit.MILLISECONDS)
        .build()
WorkManager.getInstance(this).enqueue(
    uploadMessagesRequest)
```

We use **PeriodicWorkRequestBuilder** to create a **WorkRequest** instance that runs **UploadMessagesWorker** once every week. The **WorkRequest** instance has a constraint that requires an unmetered network connection (Wi-Fi). It also specifies a linear backoff policy for retries – this means that each retry attempt is delayed by a fixed amount of time, increasing linearly with each subsequent retry.

The **enqueue()** method schedules the **WorkRequest** instance to run. If the constraints are met and there's no other work ahead of it in the queue, it will start running immediately. Otherwise, it will wait until the constraints are met and it's the **WorkRequest** instance's turn in the queue.

Please note that due to OS restrictions, a **PeriodicWorkRequest** instance may not run exactly when the period elapses; it may have some delay, but it will run at least once within that time period.

We can call this code and enqueue the **WorkRequest** instance from any place in our app, but to ensure it gets scheduled, the most convenient place is when we start up the app, in the **WhatsAppApplication.onCreate** method:

```
@HiltAndroidApp
class WhatsAppApp: Application() {
    override fun onCreate() {
        super.onCreate()
        //Include WorkRequest initialization here
    }
}
```

With all this, we would have our app ready to periodically back up messages, and our work well could have finished here. However, to explore a different approach, let's see what happens if we need to integrate another storage provider – for example, Amazon S3.

Using Amazon S3 for storage

Amazon S3 is a scalable, high-speed, web-based cloud storage service designed for online backup and archiving of data and applications on **Amazon Web Services (AWS)**. It's a well-known alternative to Firebase Storage.

Here's a brief overview of some key features and capabilities of Amazon S3:

- **Storage:** Amazon S3 can store any amount of data and access it from anywhere on the web. It provides virtually limitless storage.
- **Durability and availability:** Amazon S3 is designed for 99.999999999% (11 9s) of durability, and it stores redundant copies of data across multiple geographically separated data centers. It also provides 99.99% availability of objects over a given year.

- **Security:** Amazon S3 provides advanced security features such as encryption for data at rest and in transit, and fine-grained access controls to resources using **AWS Identity and Access Management (IAM)**, **access control lists (ACLs)**, and bucket policies.
- **Scalability:** Amazon S3 is designed to scale storage, requests, and users to support an unlimited number of web-scale applications.
- **Performance:** AWS storage makes sure that when you add or delete files, you can immediately read the latest version of your files. If you overwrite a file or delete it, there might be a short delay before these changes are fully updated everywhere.
- **Integration:** Amazon S3 integrates well with other AWS services, such as AWS CloudTrail for logging and monitoring, Amazon CloudFront for content delivery, AWS Lambda for serverless compute, and many more.
- **Management features:** S3 provides functionalities for management tasks such as organizing data and configuring finely-tuned access controls to meet specific business, organizational, and compliance requirements.
- **Data transfer:** S3 Transfer Acceleration enables fast, easy, and secure transfers of files over long distances between your client and your Amazon S3 bucket.
- **Storage classes:** Amazon S3 provides several storage classes for different types of data storage needs, such as S3 Standard for general-purpose storage of frequently accessed data, S3 Intelligent-Tiering for data with unknown or changing access patterns, S3 Standard-IA for long-lived but infrequently accessed data, and S3 Glacier for long-term archive and digital preservation.
- **Query-in-place functionality:** S3 Select enables applications to retrieve only a subset of data from an object by using simple SQL expressions.

These features make Amazon S3 a robust and versatile choice for various use cases, ranging from web applications to backup and restore, archive, enterprise applications, IoT devices, and big data analytics.

To implement our storage solution based on Amazon S3, we first need to integrate the AWS SDK into our app.

Integrating the AWS S3 SDK

We can integrate the AWS S3 SDK into our Android project by adding the following dependencies in our **build.gradle** file:

```
implementation 'com.amazonaws:aws-android-sdk-s3:
$latest_version'
implementation 'com.amazonaws:aws-android-sdk-
cognitoidentityprovider:$latest_version'
```

We have added here dependencies for the AWS SDK and the dependency needed to use Amazon Cognito.

We'll also need to provide our AWS credentials (access key ID and secret access key) to the SDK. For mobile applications, it is recommended to use Amazon Cognito for credential management.

Setting up Amazon Cognito

Amazon Cognito is a service that provides user sign-up and sign-in services, as well as access control for mobile and web applications. When you use Amazon Cognito for your user pool, you have the option to secure your data in AWS services (such as Amazon S3 for file storage) without having to embed AWS keys in your application code, which is a significant security risk.

Here are the instructions to set up Amazon Cognito in our Android application:

1. First, go to the Amazon Cognito console:

<https://console.aws.amazon.com/cognito/home>.

2. From there, click **Identity Pools**, then **Create new identity pool**.

3. Check **Guest Access** under the **Authentication** section, and click **Next**.
4. Select **Create a New IAM Role**, create a name for it, and click **Next**.
5. Then create a new name for the identity pool and click **Next**.
6. Review the summary (as in *Figure 3.4*), then click **Create identity pool**:

Amazon Cognito > Identity pools > Create identity pool

Step 1
[Configure identity pool trust](#)

Step 2
[Configure permissions](#)

Step 3
[Configure properties](#)

Step 4
Review and create

Review and create Info

Step 1: Configure identity pool trust Edit

Authentication

User access

Guest access

Step 2: Configure permissions Edit

Guest role

Guest role

WhatspacktIAM

► **View policy document**
View the policy document that Amazon Cognito has generated.

Step 3: Configure identity pool properties Edit

Identity pool name

Name

WhatsPacktIdentityPool

Basic (classic) authentication

Basic authentication

☐ Inactive

Tags (0)

Key	Value
No tags added	

Cancel Previous Create identity pool

Figure 3.4: New identity pool configuration

NOTE

Here, we are enabling access to unauthenticated identities. You also have the option to give access only to authenticated identities, but note that you will have to create every user in Amazon Cognito. Nevertheless, this approach is more secure than using the S3 SDK to store keys in our app code.

7. Next, in our app, we'll need to obtain the AWS credentials provider. For that, we will initialize **CognitoCachingCredentialsProvider** with our **IdentityPoolId** class, in the region we configured it:

```
val credentialsProvider =  
    CognitoCachingCredentialsProvider(  
        applicationContext,  
        "IdentityPoolId", // Identity Pool ID  
        Regions.US_EAST_1 // Region  
    )
```

8. Now, we can use the credentials provider instance while creating a client for the AWS service. For example, to use it with Amazon S3, use this code:

```
val s3 = AmazonS3Client(credentialsProvider)
```

Now, it is time to create a new storage provider.

Creating an AWS S3 Storage provider and integrating it into our code

Now, we need to do the same thing we did for Firebase Storage but with the AWS SDK: create a provider. This provider will be **AWSS3Provider** and will be used to handle the upload of files to AWS S3. It will take a **Context** object and a **CognitoCachingCredentialsProvider** object as constructor parameters.

This is how we can implement it:

```
class AWSS3Provider(  
    private val context: Context,  
    private val credentialsProvider:  
        CognitoCachingCredentialsProvider  
) {  
    suspend fun uploadFile(bucketName: String, objectKey:  
        String, filePath: String) {  
        withContext(Dispatchers.IO) {  
            val transferUtility = TransferUtility.builder()  
                .context(context)  
                .awsConfiguration(AWSMobileClient  
                    .getInstance().configuration)  
                .s3Client(AmazonS3Client(  
                    credentialsProvider))  
                .build()  
            val uploadObserver = transferUtility.upload(  
                bucketName,  
                objectKey,  
                File(filePath)  
            )  
            uploadObserver.setTransferListener(object :  
                TransferListener {  
                override fun onStateChanged(id: Int, state:  
                    TransferState) {  
                    if (TransferState.COMPLETED == state) {  
                        // The file has been uploaded  
                        successfully  
                    }  
                }  
                override fun onProgressChanged(id: Int,  
                    bytesCurrent: Long, bytesTotal: Long) {  
                    val progress = (bytesCurrent.toDouble()  
                        / bytesTotal.toDouble() * 100.0)  
                    Log.d("Upload Progress", "$progress%")  
                }  
                override fun onError(id: Int, ex:  
                    Exception) {
```

```

                                throw ex
                            }
                        })
                    }
                }
            }
        }
    }

```

The **uploadFile** function is a suspending function, meaning it can be called from any coroutine scope. The **withContext(Dispatchers.IO)** function is used to switch the coroutine context to the I/O dispatcher, which is optimized for I/O-related tasks, such as network calls or disk operations.

Let's delve into the **uploadFile** function, which is the core of this class.

The **TransferUtility** class simplifies the process of uploading and downloading files to/from Amazon S3. Here, we're building a **TransferUtility** instance, providing it with the Android context, AWS configuration, and an **AmazonS3Client** instance initialized with the provided **CognitoCachingCredentialsProvider** class.

The **transferUtility.upload()** method is used to upload a file to the specified bucket in S3. We provide the name of the bucket (**bucketName**), the key under which to store the new object (**objectKey**), and the file we want to upload (**File(filePath)**). This function returns a **UploadObserver** instance.

UploadObserver is used to monitor the progress of the upload.

We attach **TransferListener** to the observer to get callbacks when the upload state changes, the upload makes progress, or an error occurs.

The **onStateChanged()** method is called when the state of the transfer changes. If the state is **TransferState.COMPLETED**, it means the file has been uploaded successfully.

The **onProgressChanged()** method is called when more bytes have been transferred. Here, we calculate the progress as a percentage and log it.

The **onError()** method is called if an error occurs during the transfer. We will throw an error when it happens, to be handled by the consumers or this provider.

The **uploadFile** function is called from within a coroutine, and since the actual upload operation is a network I/O operation, it's wrapped in **withContext(Dispatchers.IO)**. This ensures the operation doesn't block the main thread, as the I/O dispatcher uses a separate thread pool that's optimized for disk and network I/O.

Now, we will need to create a data source to connect our **BackupRepository** instance to this new provider. The best way to do it is by implementing **IStorageDataSource**, a common interface for both data sources. This way, you're able to swap the underlying implementation (Firebase Storage, AWS S3, and so on) without changing the rest of your code. (This is an application of the **Dependency Inversion Principle (DIP)**, one of the SOLID principles of **object-oriented (OO)** design, which helps make your code more flexible and easier to maintain.)

This is how we will implement **S3StorageDataSource**:

```
class S3StorageDataSource @Inject constructor(
    private val awsS3Provider: AWSS3Provider
) : IStorageDataSource {
    override suspend fun uploadFile(remotePath: String,
        file: File) {
        awsS3Provider.uploadFile(BUCKET_NAME, remotePath,
            file.absolutePath)
    }
    companion object {
```

```
        private const val BUCKET_NAME = "our-bucket-name"
    }
}
```

In this code, we are implementing the **uploadFile** function calling the **awsProvider.uploadFile** function, which will upload the file to the bucket with the **our-bucket-name** name.

This new **S3StorageDataSource** class can be provided via Hilt in a similar way to the previous **FirestoreStorageDataSource** class:

```
@Module
@InstallIn(SingletonComponent::class)
object StorageModule {
    @Provides
    @Singleton
    fun provideStorageDataSource(awsS3Provider:
        AWSS3Provider): IStorageDataSource {
        return S3StorageDataSource(awsS3Provider)
    }
}
```

Here, we create a **@Module** annotation that includes a **@Provides** or **@Binds** method for **IStorageDataSource**, and Hilt will take care of injecting the right implementation based on your configuration. If you want to switch from Firebase Storage to AWS S3, you'd modify this module to provide **S3StorageDataSource** instead of **FirestoreStorageDataSource**.

Finally, we need to integrate it into our **BackupRepository** class. It is as easy as replacing the **StorageDataSource** dependency for the **IStorageDataSource** dependency:

```
class BackupRepository @Inject constructor(
    private val messageDao: MessageDao,
    private val conversationDao: ConversationDao,
```

```

private val storageDataSource: IStorageDataSource
) {
    // The rest of the class as it was before
    ...
}

```

And that's all. Depending on what we are providing in our Hilt module to satisfy the **IStorageDataSource** dependency, it will use the Firebase Storage one or the AWS S3 one.

And with this change, we finish this chapter and also our work in the WhatsApp application!

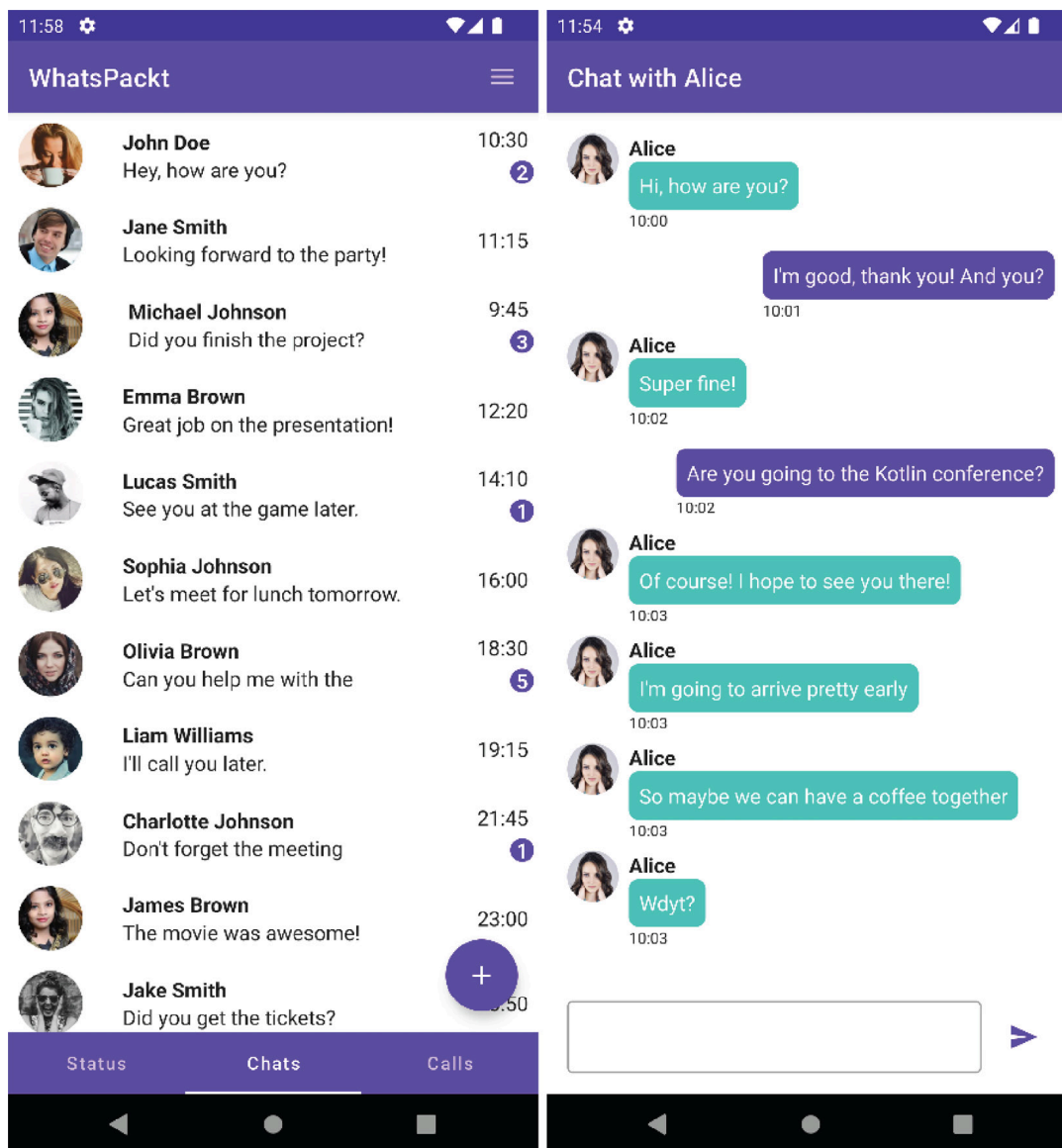


Figure 3.5: WhatsApp's final appearance

Summary

In this chapter, we centered our efforts on creating a good offline experience for our user (storing the messages in a local database using Room) and providing a mechanism to store the messages backup, in case something fails. We have also learned how to use different providers to store our files in the cloud using Firebase Firestore and AWS S3.

Now, we have finished our work in the WhatsPackt app. In the next chapter, we will start building a new app: Packtagram. It will be an app to share photos and videos with our friends that will provide new and different challenges when creating it, such as capturing video. These are challenges that we will learn to overcome.